

CS 4720 HW4

1a) It works automatically for a subset because when we talk about a belief state, we mean that an action sequence works anywhere in the set that we are actually in, which means that it will work for any subset of that set.

b) It works for a subset, but it will not work for a superset because when we are adding states, we are adding more uncertainty to it, as we don't know whether the given solution will work for the additional states or supersets of b.

c) We can modify our search by saying that in sensorless search, we don't know where we are, so we can say that each node in the search comes under a belief state. Considering the answers to parts a and b, we can improve our search results by shortening its exploration if:

- Any action can solve a larger belief set, then we can say that any subset that we might encounter in our search will be solved automatically, and it does not need to be expanded.
- We know that a smaller set can't be solved, then any superset containing it would be of no use, since the solution would fail on any problematic state, which may allow the search to avoid redundancy and reduce the size of the explored graph.

d) The AND-OR can be used over here because the actions are followed by observations. Over here, all nodes represent belief states, and after each action, the search must branch on possible observations and update the belief state accordingly. Let us strengthen our understanding by pondering over the following points, which we have discussed in the earlier part of this question:

- If a belief set is solved, then all of its subsets are also solved, and they need not be explored.
- If some belief state is not solvable, then exploring its superset would be of no use, since the solution will be on any problematic set.

Thus, by looking at the points above, we can say that the AND-OR search branches on its actions, and moreover, it also reuses these subsets or supersets relations to avoid any unnecessary expansions.

2) I have made a conditional plan for this question:

- Since we are starting from S_0 , we have 2 choices, either we can do action a_1 or do a_2 . Let us first consider the first case where we do a_1
- If we land on S_g after doing that then we can stop
- Otherwise, if we land on S_1 then we need to do a_3 and then if we land on S_g then we need to stop
- Otherwise, if we land on S_3 via a_4 then we can do action a_2 and land on S_g and stop after that.
- Now, let us consider case 2, where we take a_2 and land on S_2 , then we can do action a_4 and go to S_3 and then take a_2 to reach our goal and stop.

Here is how I have coded it:

```
if in  $S_0$  then
do  $a_1$ 
if in  $S_g$  then stop
else if in  $S_1$  then
do  $a_3$ 
if in  $S_g$  then stop
else do  $a_4$ 
if in  $S_3$  then
do  $a_2$ 
if in  $S_g$  then stop
if in  $S_2$  then
do  $a_4$ 
if in  $S_3$  then
do  $a_2$ 
if in  $S_g$  then stop
```

3a) Let us assume that S_a and S_b are a's and b's position.

Root node: (1,4)

After A's move: (2,4) (since 2 is the next empty space next to 1 for A)

After B's move: (2,3) (since 3 is the next empty space next to 4 for B)

After A's move:

Over here, we are going to have 2 cases:

1) If A moves back

Position: (1,3)

2) If A jumps over B

Position: [4,3] (A wins over here, as it reaches over here)

If case 1 prevails:

After B's move: (1,2) (since 2 is the next empty space next to 3 for B)

After A's move: (3,2) (since 3 is the next empty space to 1 for A)

After B's move:

Over here we are going to have 2 cases:

1) If B moves back

Position: [3,1] (Since A loses over here, value of A is -1)

2) If B jumps over A

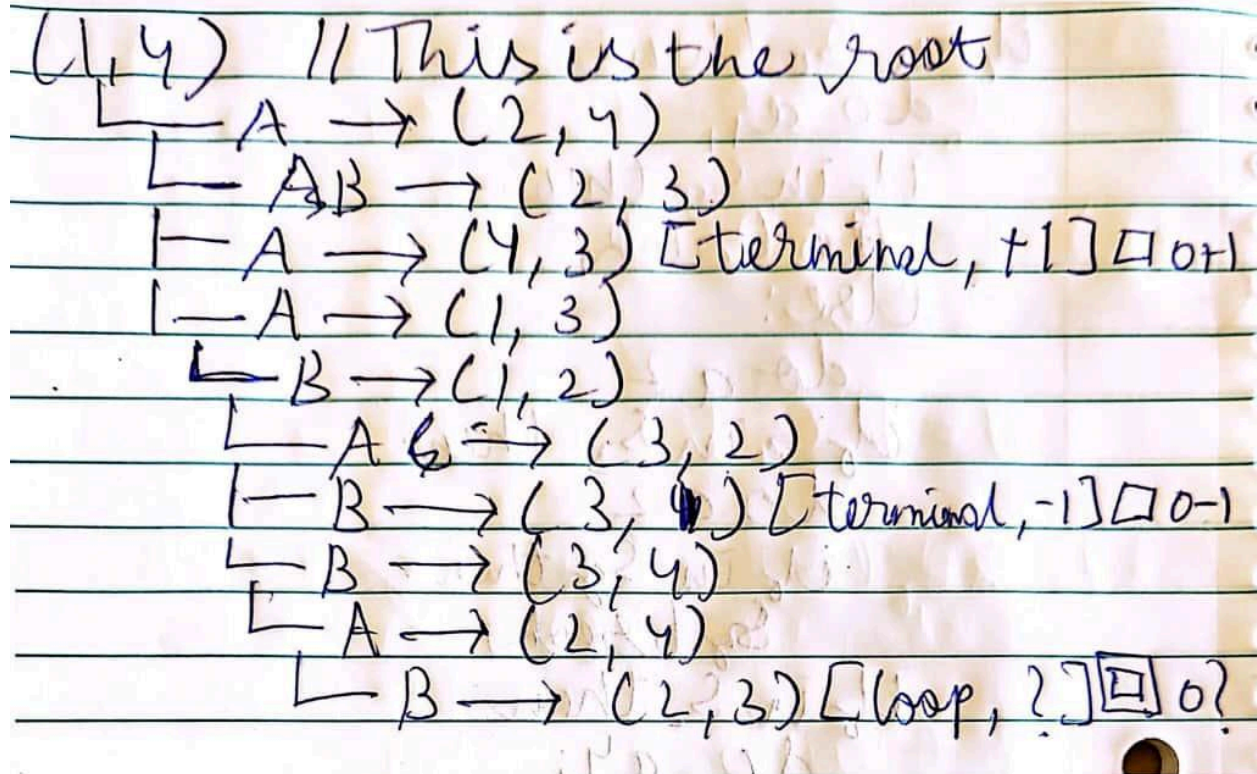
Position: (3,4)

After A's move: (2,4) (Since 2 is the next empty space for A)

After B's move: (2,3) ("?")

Since, we had already discovered this node in our second step, we can say that it is in a loop and connect it back to the earlier node rather than expanding it again, since it is a loop.

It can be represented in the following manner.



Over here, I have used single box for terminals (□), a double box for the loop state (□) and circles to represent the values (○+1 / ○-1 / ○?)

b) When we begin at $(1, 4)$ and its A's turn to move A goes to $(2, 4)$ and then B goes to $(2, 3)$ and then its A's turn to move again, so it jumps over B and reaches the opposite terminal and that's why all of these positions attain the value +1.

In case if returns back to 1, from the position $(2, 3)$ and then its B's turn to move so it attains the position $(1, 2)$, and now since its A's turn it jumps over B and goes to the position $(3, 2)$ and then after that its B's turn so it goes to $(3, 1)$ and reaches the opposite terminal and that's why it attains the value of -1.

c) Since minimax assumes a finite and acyclic tree, and our game already has a cycle $[(2, 3) \rightarrow (1, 3) \rightarrow (3, 2) \rightarrow (3, 4) \rightarrow (2, 4) \rightarrow (2, 3)]$, which contains infinite recursion as the these values repeat and repeat again and again and have no end to it, so we can say that minimax fails over here. In order to fix it, let us consider a scoreboard that contains every position where terminals have the same rules (that is, +1 if A wins and -1 if B wins), nonterminals are unknown, and we don't land in an endless loop if we come back to a state which was already discovered, then we will update A if we see any child is +1, otherwise if all the children are -1, then we will mark it as

-1. If, it is B's turn then mark the state as -1 as soon as we see any child marked as -1. If all the children are +1, then mark it as +1, otherwise keep it unknown and then we repeat these updates until anything changes.

d) Let us consider 2 cases over here:

Case 1: n is even

Let us consider a variable, say c , which can be represented as $(n+1)/2$. So, over here, we know that after each A's move, B moves to A's counterpart, which means that both of them have to maintain the same distance from their opposite terminals. Since A moves first, we know that when A reaches the position c , B can jump over it and get closer to its opposite terminal as compared to A, and then A moves to the position $n-1$, and then B eventually reaches its opposite terminal and wins the game.

Ex: Let us consider that $n = 5$, and A moves first and goes to occupy the position (2,5) and then B moves to A's counterpart and occupies the position (2,4) and then A moves to (3,4) (which is $(n+1)/2$ as $(5+1)/2 = 3$). After that B jumps over A occupies the position (3,2) and comes more closer to its opposite terminal as compared to A and then moves to (4,2) and then eventually B plays the winning move as it moves to the position (1,4).

Case 2: n is odd

Again, since A moves first and then B moves to its counterpart, it means that again, both of them maintain the same distance from their opposite terminals and once A and B reach the positions $n/2$ and $n/2 + 1$ respectively, we know that A can jump over B and come more closer to its opposite terminal as compared to B and eventually it goes to win the game.

Ex: Let us consider that $n = 6$, and A moves first and goes to occupy the position (2,6), and then B moves to A's counterpart and occupies the position (2,5), and then A moves to (3,5). After that B moves onto the position (3,4) (over here A and B occupy the positions $n/2$ and $n/2 + 1$) and then A jumps over B and moves onto (5,4) comes more closer to its opposite terminal as compared to B and then B moves to (5,3) and then eventually A plays the winning move as it moves to the position (6,3).